

The Alberta Buck - Notes Mutual Decryptability (DRAFT v0.1)

Closing the Issuer-Binding Gap so Every BUCK Note Yields a Provable Receipt

Perry Kundert

2026-05-29



A monetary system can promise privacy or it can promise accountability; the Alberta Buck promises both at once, along a single seam. *Bulk* traceability is defeated – no chain-analytics firm can reconstruct who paid whom – while *bilateral* disclosure is preserved: the two parties to any payment always retain the cryptographic means to prove, under compulsion, exactly who the counterparty was. The load-bearing half of that promise is the **non-deniable-receipt invariant**: there must be no path by which someone pays BUCKs to another and keeps cryptographic deniability of the act. Concretely, a recipient who is compelled by a court, and who chooses to comply, must be able to produce a cryptographically sound receipt naming the payer's *decrypted, KYC-bound Identity*.

This document shows that BUCK Notes do not yet satisfy that invariant, defines it formally, and specifies the fix. The shipped spend path verifies the *depositor's* identity (`IdentityRegistry.verifySpendCP`), but the mint path verifies *nothing* about the *issuer's*: the per-leaf `issuerData` is folded into the commitment's `idHash` and never checked – exactly the "flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2)" that `mint.circom` marked "deferred to a later revision" and that was never re-introduced. A dishonest or colluding issuer can therefore mint a note that pays a recipient while leaving no recoverable payer Identity. The remedy is to make the *deferred approve handshake* symmetric and fully verified: bind a checked issuer identity at mint (a Schnorr binding for public issuers; a recipient-hiding re-encryption proof for the private A2 issuer), and add the `E_depositor_for_issuer` payload to B-note deposits. Each leg is stated as a theorem with an executable `py_ecc` check, and the circom / EVM formal-verification tooling that can mechanize the soundness arguments is catalogued. (Notes, Proofs)

Contents

1	The Non-Deniable-Receipt Invariant	4
1.1	Statement	4
1.2	Why the asymmetry between flavors	4
2	The Receipt as a Proof Chain	5
3	What the Shipped Implementation Binds	6
4	The Gap, Per Flavor	7
4.1	A2 – addressed, private issuer	7
4.2	A1 – addressed, public issuer	7
4.3	B1 – bearer, public issuer	7
4.4	Summary	7
5	The Fix: A Symmetric, Verified Handshake	8
5.1	Issuer half, bound at mint	8
5.2	Depositor half, at deposit	8
5.3	Enforcing bearer $\Rightarrow$ public issuer	9
6	Protocol Changes (Implementation Surface)	10
6.1	IdentityRegistry	10
6.2	Notes.sol	10
6.3	Circuits	10
6.4	Wallet (alberta_buck.wallet)	11
7	Provable Correctness	12
7.1	Theorems	12
7.2	Executable sanity checks (py_ecc on BN254)	12
8	Formal-Verification Approaches	14
8.1	The cryptographic mathematics	14
8.2	The circom circuits (soundness is the hard part)	14
8.3	The Solidity verifiers and Notes.sol	14
8.4	What "provably correct per step" buys	15
9	Test Plan	16
9.1	Positive	16
9.2	Negative (these must now fail, and did not before)	16
9.3	Cross-artifact	16
10	Phasing and Migration	17
11	Implementation Status (Phase 1 shipped)	18
11.1	Phase 1 – shipped, wired, and tested	18
11.2	Decisions / deviations from the spec text above	18
11.3	Residual gaps $\rightarrow$ Phase 2 (NOT done)	19
11.4	Cross-references and house conventions	19

DRAFT

1 The Non-Deniable-Receipt Invariant

The Alberta Buck identity layer is built so that *every* BUCK transfer leaves both endpoints holding decryptable proof of the other's Identity. For a direct **transfer**, the bilateral **approve** handshake establishes this: each party re-encrypts their KYC-certified credential under the other's key and proves (Chaum-Pedersen) that the re-encryption carries the same Identity point M . BUCK Notes defer and reorganise that handshake through the commitment pool, and the invariant must survive the reorganisation.

1.1 Statement

Definition 1. Let an Identity be the KYC-certified point $M \in \mathbb{G}_1$ bound to a registered account by an issuer's Pointcheval-Sanders credential and stored (ElGamal-encrypted) as $E_{addr}[X] = (R_X, C_X) = (r_X G, M_X + r_X \cdot pk_X)$, with $pk_X = sk_X G$. A payment moves value v from a payer P to a recipient D .

Definition 2. (Non-deniable receipt.) A protocol satisfies the **non-deniable-receipt invariant** if for every accepted payment there exists a receipt $\mathcal{R}$ and an efficient public predicate RcptVerify such that:

- (**Completeness.**) The recipient D can compute $\mathcal{R}$ from data they hold plus public chain state, and $\text{RcptVerify}(\mathcal{R}) = 1$ names a registered Identity M_P and value v .
- (**Soundness / non-frameability.**) No PPT adversary can produce $\mathcal{R}'$ with $\text{RcptVerify}(\mathcal{R}') = 1$ naming an Identity $M_{P'}$ that did not author the payment.
- (**No deniability.**) No PPT payer can cause an accepted payment to D for which no accepting receipt naming the payer exists – even in collusion with D .

The third clause is the operative one: it is the collusion-resistant form. A self-interested recipient who simply wants a record is protected by completeness alone; the invariant additionally forbids a *willing* pair from constructing an untraceable BUCK payment.

1.2 Why the asymmetry between flavors

Who can be named, and how, depends on what is known at mint time – which is exactly the axis that defines the note flavors:

- **B (bearer).** The depositor is unknown at mint, so the note cannot carry a recipient-targeted ciphertext. The *issuer* must therefore be a **public** Identity: their M is in the clear and the recipient names them directly. A bearer note from an *encrypted* issuer is unnameable and must be impossible to mint.
- **A (addressed).** The issuer knows the recipient at mint and has already re-encrypted material the eventual depositor can open. The issuer may stay *private* (A2) by re-encrypting their *own* Identity under the recipient's key ($E_{\text{iss-for-rec}}$), which only the recipient can decrypt, or *public* (A1) by attaching it in the clear with a signature.

In all cases the recipient must end up holding a path to a *verified* issuer Identity. The next section shows the shipped code does not provide one.

For the recipient to prove "I received v BUCKs issued by Identity M_P ", the receipt must chain four links, each independently checkable against public chain state:

- Links (a), (b), (d) are already sound in the shipped contracts. (b) holds because `cms []` live in the mint transaction’s `calldata`, which is immutable chain history; the recipient reveals `cm` to anchor it (accepting the `spend`↔`mint` linkage, which is the price of complying). (d) holds because the nullifier is a deterministic function of the opening and is emitted by `Notes.Spent / Notes.SpentA` alongside the ERC-20 `Transfer` from the pool.

The invariant lives or dies on link (c) – and (c) is precisely the step the implementation never enforces.

3 What the Shipped Implementation Binds

The spend side verifies the depositor's Identity; the mint side verifies nothing about the issuer's. This asymmetry is the whole bug.

Leg	Shipped check	Guarantee
Depositor (A-spend)	<code>IdentityRegistry.verifySpendCP(spender, recipient, E_n, pi)</code>	spender provably <i>is</i> the addressed M_{rec}
Depositor (B-spend)	none (knowledge of ρ is the authorization)	spender is <i>some</i> registered recipient
Issuer (any mint)	none – <code>idHash</code> folds <code>issuerData</code> via Poseidon-8, value never checked	none

Concretely, from the source:

- `circuits/spend_a.circom` constrains `idHash === Poseidon8(eNoteR.x, eNoteR.y, eNoteC.x, eNoteC.y, issuerData[0..3])`, where `issuerData` is a *private witness* holding A1 ($m_{iss}, \sigma_R.x, \sigma_R.y, \sigma_s$) or A2 (R_{iss}, C_{iss}) = $E_{iss-for-rec}$. The circuit *hashes it in* but never verifies the signature or the encryption.
- `circuits/spend.circom` (the shipped B-spend) leaves `flavor` unconstrained and binds *no* issuer material at all; `idHash` is opaque.
- `circuits/mint_batch.circom` public inputs are `[oldRoot, newRoot, nextLeafIndex, totalFace, cm]` with private `[flavor, v, rho, idHash, predicate, siblings]`. No issuer key, no signature, no ElGamal well-formedness. `mint.circom` says so in as many words: *"flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2) is deferred to a later revision."*
- `src/Notes.sol mint()` records `Minted(issuer = msg.sender, ...)` but ties the minting account to the leaves only through transaction authorship, never cryptographically to the committed `issuerData`.

4 The Gap, Per Flavor

4.1 A2 – addressed, private issuer

The opening carries $E_{\text{iss-for-rec}} = (R_{\text{iss}}, C_{\text{iss}})$, an ElGamal ciphertext the recipient decrypts with sk_{rec} to obtain M_{iss} . It is bound into `idHash` but **not** proven to re-encrypt the issuer’s *registered* credential. A malicious issuer commits a random $(R_{\text{iss}}, C_{\text{iss}})$: the note mints and spends cleanly, the recipient decrypts to a meaningless point, and `link(c)` yields no registered Identity. **The payer keeps full deniability.**

4.2 A1 – addressed, public issuer

The opening carries (m_{iss}, σ) . The Schnorr signature σ is never verified, and nothing forces m_{iss} to equal the *minting* account’s registered M . A malicious issuer can omit a valid σ (no receipt), or commit a *victim’s* m_{iss} and a forged σ (frame), and the note still works.

4.3 B1 – bearer, public issuer

`spend.circom` binds no issuer Identity whatsoever. The only issuer signal is `msg.sender` in the `Minted` event. A receipt is therefore constructible *only if* the issuer minted from a **public** Identity – and the contract cannot enforce that, because `cms[]` are opaque and it cannot tell a bearer leaf from an addressed one. A bearer-spendable note minted from an *encrypted* Identity is unnameable. **The payer keeps full deniability.**

4.4 Summary

In every flavor the issuer Identity is, at best, *committed* but never *verified*; in B1 it is not even committed. The depositor half of the handshake is checked (`verifySpendCP`); the issuer half is missing. The fix is to add it.

5 The Fix: A Symmetric, Verified Handshake

Restore the symmetry the "deferred approve handshake" framing always implied: the *mint* is the issuer's half of approve and must be *verified*, exactly as the *deposit* verifies the depositor's half.

5.1 Issuer half, bound at mint

At mint, the issuer is `msg.sender` – *not* anonymous (they authored the tx). So the issuer's registered credential $E_{\text{addr}}[\text{issuer}]$ may be read as a public input; only the *recipient* (for A2) must stay hidden.

- **Public issuer (A1, B1).** Require `msg.sender` to be `isPublicIdentity`, and verify a Schnorr signature by their registered key pk_{iss} over the batch commitment $H_{\text{batch}} = \text{keccak}(\text{cms})$. Bind $m_{\text{iss}} = M_{\text{iss}}$ (the in-the-clear registered Identity) into `idHash`. Because the registered Identity of a public account is in the clear, link (c) reduces to a registry read plus the verified signature.
- **Private issuer (A2).** The issuer proves `E_iss-for-rec` re-encrypts their *own* registered M_{iss} under the recipient's key pk_{rec} – the `verifyApprove` relation with `sender = issuer`, `spender = recipient`. But `verifyApprove` as shipped takes pk_{rec} as a verifier input and so *names the recipient*, which A2 must not do. Two recipient-hiding realisations:

1. **In-SNARK (mint circuit).** Extend `mint_batch.circom` so each A2 leaf proves, with pk_{rec} and M_{iss} as private witnesses:

$$pk_{\text{iss}} = sk_{\text{iss}}G, \quad M_{\text{iss}} = C_{\text{iss}}^{\text{reg}} - sk_{\text{iss}}R_{\text{iss}}^{\text{reg}}, \quad E_{\text{iss-for-rec}} = (r'G, M_{\text{iss}} + r'pk_{\text{rec}}),$$

with $E_{\text{addr}}[\text{issuer}] = (R_{\text{iss}}^{\text{reg}}, C_{\text{iss}}^{\text{reg}})$ a *public* input and pk_{rec} the same key that re-randomises the note ciphertext E_{note} . Cost: a handful of in-circuit $\mathbb{G}_1$ scalar muls per A2 leaf – the ~10x the original deferral feared, but bounded and paid only on A2 mints.

2. **Recipient-blinded on-chain CP.** Keep verification on-chain (EIP-196, cheap) but blind pk_{rec} : the issuer proves the re-encryption relation against a *Pedersen commitment* to pk_{rec} that also appears (identically committed) in E_{note} , so the verifier never learns pk_{rec} . This is a new sigma protocol (Section *Provable Correctness*), avoiding in-SNARK $\mathbb{G}_1$ arithmetic at the cost of a bespoke proof.

The in-SNARK route (1) is the recommended default: it reuses the trusted setup already in place for the batch mint and keeps the on-chain mint a single verifier call.

5.2 Depositor half, at deposit

- **A1, A2.** Already shipped: `verifySpendCP` proves the spender is the addressed M_{rec} . The issuer, who chose M_{rec} at mint, already holds the symmetric knowledge.
- **B1.** Add `E_depositor_for_issuer` to the spend: the depositor re-encrypts *their* registered M_{dep} under the public issuer's pk_{iss} , with a `verifyApprove`-style CP proof (`sender = depositor`, `spender = issuer`, both public-or-named here so no blinding is needed), emitted in the `SpentB` event. This closes the symmetric direction – the public bearer issuer can identify who cashed their note.

5.3 Enforcing bearer $\Rightarrow$ public issuer

The contract cannot read `flavor` from an opaque `cm`. Make the mint flavor-aware for the binding: the mint circuit exposes, per leaf, a public `issuerMode` in `{PUBLIC, PRIVATE}` and proves the matching binding (Schnorr-bound public M , or in-SNARK re-encryption). `Notes.mint` requires every `PRIVATE` leaf to be an A-flavor (addressed) leaf – a bearer (B) leaf with `issuerMode = PRIVATE` is unsatisfiable – and requires `msg.sender` to be `isPublicIdentity` whenever any `PUBLIC` leaf is present.

DRAFT

6 Protocol Changes (Implementation Surface)

6.1 IdentityRegistry

```
// Public-issuer binding: Schnorr by msg.sender's registered key over H_batch.
function verifyIssuerSchnorr(address issuer, bytes32 hBatch, SchnorrProof calldata s)
    external view returns (bool);

// Private-issuer binding (A2), recipient-blinded sigma variant (option 2):
// proves E_iss_for_rec re-encrypts E_addr[issuer]'s M under a *committed* pk_rec
// that the mint SNARK also binds to E_note, without revealing pk_rec.
function verifyIssuerCPBlinded(address issuer, ElGamalCT calldata eIssForRec,
    bytes32 pkRecCommit, IssuerCPPProof calldata pi)
    external view returns (bool);

// B-note depositor->issuer (mirror of verifyApprove, both parties named):
function verifyDepositorForIssuer(address depositor, address issuer,
    ElGamalCT calldata eDepForIss, CPPProof calldata pi)
    external view returns (bool);
```

Under the in-SNARK route the A2 binding is discharged inside the mint verifier and `verifyIssuerCPBlinded` is unnecessary; the registry then only gains `verifyIssuerSchnorr` and `verifyDepositorForIssuer`.

6.2 Notes.sol

- `mint(...)` gains per-leaf `issuerMode[]` (public input of the mint SNARK) and, for public leaves, a `SchnorrProof` over `keccak(cms)`; it requires `msg.sender` public when any public leaf is present, and rejects `PRIVATE+bearer` leaves. The A2 re-encryption binding rides inside the mint proof (option 1) or is checked per leaf via `verifyIssuerCPBlinded` (option 2).
- `spend(...)` (B) gains `(E_depositor_for_issuer, CPPProof)`, calls `verifyDepositorForIssuer(msg.sender, issuer, ...)`, and emits `SpentB(nullifier, face, recipient, eDepForIss)`. The issuer address is recovered from the leaf's public-issuer binding revealed at `spend`.
- `spendACP(...)` (A) is unchanged on the depositor side; the issuer guarantee now comes for free from the mint-time binding.

6.3 Circuits

- `mint_batch.circom`: add the per-leaf `issuerMode` public signal and, for A2 leaves, the re-encryption constraints above (option 1). For public leaves, constrain `idHash`'s `issuerData` prefix to equal the registered public `M` so the in-clear identity is the one bound.
- `spend.circom` (B): constrain `flavor == B1=` and reveal enough of the leaf's public-issuer binding to recover the issuer address on-chain.
- `spend_a.circom`: unchanged (the A2 issuer binding moved to mint).

6.4 Wallet (`alberta_buck.wallet`)

- Mint: construct the issuer Schnorr proof (public) or the A2 re-encryption witness (private), mirroring `notes.id_hash_a1/a2`.
- Spend (B): construct `E_depositor_for_issuer` and its CP proof.
- Receipt: implement `RcptVerify` as a standalone verifier (opening + mint-tx reference + spend-tx reference + issuer recovery) so a recipient can produce and a third party can check the receipt entirely off-chain from chain data.

DRAFT

7 Provable Correctness

We extend the companion *Proofs* document (Theorems 1-11) with the issuer-binding results. Each is accompanied – in the proofs document’s house style – by an executable `py_ecc` sanity check on BN254.

7.1 Theorems

Theorem 3. (Issuer-binding completeness.) *An honestly generated public-issuer Schnorr proof, and an honestly generated A2 re-encryption proof (in-SNARK or blinded), verify.*

Theorem 4. (Issuer-binding soundness.) *If `verifyIssuerSchnorr` accepts, the batch commitments were authorised by `issuer`’s registered key. If the A2 re-encryption proof accepts, $E_{iss-for-rec}$ decrypts under sk_{rec} to the issuer’s registered M_{iss} (the same M certified by the issuer’s PS credential – Theorem 6). Reduction: special soundness of the underlying sigma protocol (two accepting transcripts sharing commitments extract sk_{iss}, r'), as in Theorems 4, 6; or Groth16 knowledge-soundness for the in-SNARK route.*

Theorem 5. (Receipt completeness.) *For every note accepted by the patched `mint` and redeemed by `spend` $\neq$ `spendACP`, the recipient can compute $\mathcal{R}$ with $RcptVerify(\mathcal{R}) = 1$ naming a registered Identity. Proof: links (a),(b),(d) hold as before; (c) now holds by issuer-binding soundness (public: registry $M + Schnorr$; private: decrypt $E_{iss-for-rec}$, validity guaranteed by the accepted binding).*

Theorem 6. (Receipt soundness / non-frameability.) *No PPT adversary produces an accepting $\mathcal{R}'$ naming $M_{P'}$ that did not author the payment. Reduction: a forged public receipt breaks Schnorr EUF-CMA; a forged private receipt breaks re-encryption soundness (Theorem 7) or the binding hash (collision resistance of Poseidon-8 in `idHash`).*

Theorem 7. (No deniability under collusion.) *No payer-recipient coalition can produce an accepted payment with no accepting receipt: `mint` rejects any leaf lacking a valid issuer binding, so every spendable leaf carries one, and the binding is recoverable by the holder of the opening.*

Theorem 8. (Recipient privacy preserved.) *The A2 issuer binding is zero-knowledge in pk_{rec} and M_{iss} : the in-SNARK route reveals only the Groth16 proof; the blinded route reveals only a hiding Pedersen commitment to pk_{rec} . No observer learns the recipient. Reduction: HVZK of the sigma protocol (Theorem 5) / Groth16 zero-knowledge.*

Corollary 9. (Mutual decryptability.) *Combining the issuer half (above) with the depositor half (`verifySpendCP` for A; `verifyDepositorForIssuer` for B), both parties can recover the other’s registered Identity, and each can prove it under compulsion. This is the Notes specialisation of the BUCK mutual-decryptability invariant (Proofs Theorem 11), now enforced rather than assumed.*

7.2 Executable sanity checks (py_ecc on BN254)

Mirroring the proofs document, each theorem ships a small witnessed check that runs in CI alongside the existing `alberta_buck.wallet` vector generation:

- **Issuer Schnorr:** sign H_{batch} under sk_{iss} , assert verify; assert a tampered H_{batch} or wrong key fails.

- **A2 re-encryption:** build $E_{\text{addr}}[\text{issuer}]$, decrypt to M_{iss} , form $E_{\text{iss-for-rec}}$ under pk_{rec} , run the prover/verifier; assert the recipient's sk_{rec} -decryption equals M_{iss} ; assert a random $E_{\text{iss-for-rec}}$ is rejected (soundness witness) and that the transcript hides pk_{rec} (two runs with distinct pk_{rec} are indistinguishable to the verifier given the commitment).
- **Receipt:** assemble $\mathcal{R}$ from a synthetic mint+spend and assert $\text{RcptVerify} = 1$; assert a frame attempt (swap M_P) fails.

These vectors double as the canonical inputs the Solidity and circom tests load via `vm.readFile`, keeping the on-chain verifier, the circuit, and the reference math in lock-step (the pattern already used for `verifySpendCP`).

DRAFT

8 Formal-Verification Approaches

"Every step provably correct" spans three artifact classes – the sigma-protocol mathematics, the circom circuits, and the Solidity verifiers – each with its own tractable tooling.

8.1 The cryptographic mathematics

- **Hand proofs** + `py_ecc`: the existing *Proofs* methodology – completeness, special-soundness (2-extractability), and HVZK arguments, each pinned by an executable BN254 check. Low-tech, high-confidence, already in the repo.
- **EasyCrypt**: a proof assistant purpose-built for *game-based* cryptographic proofs; the sigma-protocol soundness and ZK lemmas can be mechanised as reductions, giving machine-checked versions of Theorems above.
- **CryptoVerif** (computational) or **ProVerif** (symbolic): protocol-level analysis of the full handshake (mint binding + deposit binding) against the non-deniability and recipient-privacy goals stated as queries.

8.2 The circom circuits (soundness is the hard part)

The danger in arithmetic circuits is *under-constraint* – a witness slot that the prover can set freely, silently widening soundness (the exact failure mode of the unverified `issuerData`). Tooling that targets this directly:

- **Picus** (Veridise): automated detection of under-constrained signals; run it on the patched `mint_batch.circom` to prove the new issuer-binding signals are *determined* by the public inputs.
- **Circomspect** (Trail of Bits): static analysis / linting for common circom pitfalls (unconstrained signals, `<--` without `<=`, `=` signal-vs-constraint confusion).
- **ecne** (0xPARC): proves R1CS systems compute deterministic functions of their inputs – a soundness witness for the binding constraints.
- **Coda** / **circom-to-Coq** efforts: full functional-correctness extraction to a proof assistant for the security-critical templates (the re-encryption gate).
- Discipline: keep the security-critical constraints (the $\mathbb{G}_1$ relations) in small, separately-audited sub-templates with explicit `Num2Bits` range checks, as the value-conservation bound already does.

8.3 The Solidity verifiers and Notes.sol

- **Halmos** (a16z): symbolic-execution property tests in Solidity – assert, for all inputs, e.g. "mint reverts unless every leaf carries a valid issuer binding" and "spend emits a recoverable issuer for every accepted note."
- **Kontrol** / **KEVM** (Runtime Verification): K-framework formal proofs over EVM bytecode for the highest-assurance properties (the binding-or-revert invariant, nullifier non-reuse).

- **Certora Prover:** CVL rules for state invariants (`noteFaceSum` conservation, "no accepted spend without a prior valid mint binding").
- **solc SMTChecker:** lightweight, in-compiler assertions and overflow checks on the new code paths.
- **Foundry:** fuzz + stateful invariant testing as the always-on backstop; negative tests below are expressed here first.

8.4 What "provably correct per step" buys

The chain of guarantees is: `=py_ecc=`/EasyCrypt prove the *math*; Picus/ecne prove the *circuit* faithfully and exhaustively enforces that *math*; Halmos/Kontrol prove the *contract* admits no spend whose mint lacked the binding. Composed, no accepted Note payment can exist without a recoverable, sound receipt – the invariant, mechanically checked end-to-end.

DRAFT

9 Test Plan

9.1 Positive

- A1/A2/B1 honest mint + spend; recipient runs `RcptVerify` and a third party re-checks it – all return 1 naming the correct registered Identity.
- A2 recipient privacy: two mints to distinct recipients are observationally indistinguishable on chain.

9.2 Negative (these must now fail, and did not before)

- A2 issuer commits a random `E_iss-for-rec`: `mint reverts` (binding fails).
- A1 issuer commits a victim's m_{iss} / a forged σ : `mint reverts`.
- Bearer leaf with `issuerMode = PRIVATE`, or any public leaf from a non-`isPublicIdentity` minter: `mint reverts`.
- Recipient attempts a forged receipt naming a non-issuer: `RcptVerify = 0`.

9.3 Cross-artifact

- The `py_ecc` vectors drive both the circom witness tests and the Solidity verifier tests, asserting byte-identical `idHash` / proof encodings (the established `alberta_buck.wallet -> vm.readFile` pattern).
- Picus run in CI gates merges on "no new under-constrained signals."

10 Phasing and Migration

1. **Public issuers first (A1, B1).** Schnorr binding + `isPublicIdentity` enforcement + `verifyDepositorForIssuer` for B. No new circuit setup; pure contract + wallet work. Closes the gap for the public-issuer flavors, which are the audit-sensitive payroll / cashier-cheque cases.
2. **Private issuer (A2).** The in-SNARK re-encryption binding: extend `mint_batch.circom`, re-generate the per-N verifiers and trusted setup, wire `issuerMode`. The larger lift; ship behind a governance-set verifier slot so the new mint verifier can be rotated in (`Notes.setMintVerifier`).
3. **Mechanisation.** Land Picus/Halmos in CI; mechanise the sigma proofs in EasyCrypt as time allows.

Notes minted under the old (unbound) verifier remain spendable but do not carry the guarantee; governance announces a cutover root after which only bound mints are accepted, and wallets surface a `UNVERIFIED ISSUER` status for pre-cutover notes (a new wallet status alongside `VALID` `!=DEAD` `!=FAKE`).

DRAFT

11 Implementation Status (Phase 1 shipped)

This section records what is **actually shipped** against the spec above, so the state is self-contained. Work proceeds on branch `feature/decryptability` (cut from `master` after the `feature/buck-negative-balance` merge, `f445905`).

11.1 Phase 1 – shipped, wired, and tested

The public-issuer half and the bearer depositor->issuer half are implemented end-to-end. Commits, in order:

- `d8dc818` – `IdentityRegistry.verifyIssuerSchnorr + SchnorrProof` struct + `_fsIssuerSchnorr` helper. Checks $s * G = R + e * pk_{iss}$, gated on `isPublicIdentity[issuer]`; Fiat-Shamir binds $(pk_{iss}, R, hBatch, issuer, chainid)$.
- `0df3e71` – `Notes.mint` split into a 6-arg (legacy) and 7-arg (+ `SchnorrProof`) overload over an internal `_mint`. When `isPublicIdentity(msg.sender)`, `_mint` **requires** `verifyIssuerSchnorr(msg.sender, keccak256(cms), sig)`; the 6-arg path passes a zero proof and so reverts for a public issuer. New `IssuerBound(issuer, newRoot)` event.
- `5f92cdc` – wallet `alberta_buck/wallet/schnorr.py` (`issuer_schnorr_sign/verify, batch_commitment = keccak of packed cms`); `vectors.py` emits an `issuer_schnorr` section into `test/vectors/identity.json`; the FS transcript matches `_fsIssuerSchnorr` byte-for-byte.
- `a4b36ba` – `IdentityRegistry.verifyDepositorForIssuer` (extracted the `verifyApprove` body into `_verifyApprove`; the alias just relabels sender/spender as depositor/issuer). `Notes.spend` split into a 5-arg (legacy) and 8-arg (+ `issuer, eDepForIss, CPProof`) overload over `_spend`; the 8-arg path checks `verifyDepositorForIssuer(recipient, issuer, ...)` and emits `SpentB`.

Tests – all green; full forge suite **383 passed, 1 skipped** (the offline-skipped `BuckKControllerForkTest`):

- `test/IssuerSchnorr.t.sol` – 7 in-Solidity (completeness + soundness: tampered batch / response, wrong key, non-public, unregistered, cross-issuer replay) + 4 `IssuerSchnorrVectorTest` (the python-emitted proof verifies on-chain – the cross-artifact parity check).
- `test/NotesIssuerBinding.t.sol` – 5 (valid binding mints; tampered / wrong-batch sig and the 6-arg overload revert `Notes: bad issuer binding`; non-public minter skips).
- `test/NotesDepositorBinding.t.sol` – 5 (registry parity reusing the `approve` vector as a depositor->issuer proof; bound spend emits `SpentB`; bad CP reverts `Notes: bad depositor binding`; legacy 5-arg unbound works).
- `alberta_buck/test/test_schnorr.py` – 6 (round-trip, tamper, keccak parity).

11.2 Decisions / deviations from the spec text above

- **The Schnorr binding is at mint, gated only on the minter being public.** There is NO per-leaf `issuerMode public` signal yet, so the "Enforcing bearer > public issuer" subsection is **not** implemented: Phase 1 binds **public minters only**; the contract still cannot tell a bearer leaf from an addressed one (=cms opaque).

- `verifyDepositorForIssuer` reuses the `verifyApprove` relation verbatim – no new sigma protocol, no new wallet signer, no new vector. The existing `approve` (Alice->Bob) vector **is** a valid `depositor(Alice)->issuer(Bob)` proof. FS binds (depositor, issuer, chainid); the specific note is tied in only by `SpentB` emitting (nullifier, `eDepForIss`) atomically – the nullifier is NOT in the CP transcript.
- `Notes.spend` binds the recipient (the beneficiary), not `msg.sender`.
- Both bindings are optional overloads (non-breaking): the 6-arg `mint` and 5-arg `spend` still serve legacy / non-public paths.
- The shipped event is `SpentB(nullifier, face, recipient, issuer, eDepForIss)` (issuer included). The depositor **names** issuer as a parameter; it is **not** recovered from the leaf, because the B-spend circuit does not reveal it.

11.3 Residual gaps -> Phase 2 (NOT done)

1. **Bearer-from-non-public is still unbound** – needs the mint SNARK to expose a verified per-leaf `issuerMode` so `Notes.mint` can reject a bearer leaf from a non-public minter.
2. **The depositor names the issuer at B-spend** (not cryptographically tied to the note) – sound only for an honest depositor; binding the issuer into the B-spend circuit removes this.
3. **A2 private-issuer in-SNARK re-encryption binding** – the central Phase 2 item; extend `mint_batch.circom` + regenerate per-N verifiers / trusted setup, behind `Notes.setMintVerifier`.
4. `RcptVerify` off-chain receipt verifier (wallet) – unimplemented.
5. **Formal verification** (Picus / Halmos / EasyCrypt) – not started.

11.4 Cross-references and house conventions

- `alberta-buck-notes.org` was updated this session: the "Mutual Decryptability" invariant gained an "assumed vs. enforced" note, the "Note Flavors" intro gained a "why bearer forces a public issuer – and how it is enforced" paragraph, and Flavor B1 gained a Phase-1 status note. That `.org` edit and the regenerated PDF/TXT exports are **uncommitted**; the Phase 1 code/tests above are committed.
- Conventions in force: a negative signed BUCK balance is **not** "debt" – it is "used credit" / an outstanding claim on the holder's own assets; the carrying transfer is the `isCarrying-dispatched Buck._carryingTransfer (buckSeconds apportionment)`, **not** a `transferCarrying` primitive. Build and test via `make nix-test`; Python via `nix develop --command python -m pytest`. Commit messages: ASCII only, no co-author/AI attribution.

12 Open Questions

1. **In-SNARK vs blinded-CP for A2.** The in-SNARK route is simpler to reason about (one verifier) but enlarges the trusted setup; the blinded-CP route keeps the SNARK small but adds a bespoke sigma protocol to audit. The recommendation is in-SNARK, but the blinded-CP construction is worth a full soundness/ZK write-up before deciding.
2. **Batch vs per-leaf issuer signature.** A single Schnorr over `keccak(cms)` is cheapest but binds the *whole batch* to one public issuer; mixed-issuer batches need per-leaf signatures or per-issuer sub-batches.
3. **Receipt non-repudiation scope.** `RcptVerify` proves the issuer *authored* the note; tying that to a real-world legal identity still rests on the KYC binding of *M* (the PS credential). This document assumes that binding; it does not re-derive it.
4. **Predicate-bearing notes.** The `predicate` field is out of scope here; a programmable predicate that itself moves value would need its own receipt analysis.

DRAFT